

Executive Summary

We surveyed the state-of-the-art in guitar strumming detection and propose a multi-stage solution for your VGT project. Traditional onset-based methods (e.g. spectral flux) can find strum *events* with moderate accuracy (~75–80% F1 [35†L409-L417]), but struggle with up vs. down direction. Recent deep-learning models (especially CRNNs) trained on large synthetic+real datasets achieve much higher accuracy (~98% F1 on isolated pickups and ~92% on real mic recordings [35†L409-L417] [35†L468-L477]). We recommend a hybrid pipeline: use **signal-processing onset detectors** to generate candidate strum times (for initial labeling or inference speed), combined with **ML models** (CNN/CRNN or Transformer-based) to refine onset detection and classify down/up direction. Feature inputs like log-Mel spectrograms, possibly with harmonic/percussive separation, are effective. Training can leverage both synthetic data (via guitar MIDI/tab synth engines, as in SynthTab [37†L92-L99]) and a modest real dataset (e.g. recorded strumming with annotation). Key features include spectral flux onsets, transient/energy cues, and time-domain envelopes. State-of-art ML uses CNN+RNN or transformer encoders to model temporal context and achieve ~90–95% accuracy [35†L409-L417] [45†L38-L45] .

Practically, integrate this with your pipeline by processing the separated guitar stem: compute onset features or spectrogram frames, run the chosen model, then quantize detected strums to the known tempo grid. Output can be a MIDI track (e.g. mapping down/up to specific notes) or Reaper markers. We propose an implementation plan (see timeline diagram) starting with quick rule-based baselines and moving to ML training, using augmentation (reverb, noise, pitch-shift [32†L278-L287] [32†L295-L299]) to improve robustness. Evaluation uses MIR onset metrics (precision/recall/F1 with 50ms tolerance). We tabulate method comparisons and datasets below. Diagrams illustrate a conceptual pipeline and project timeline.

Sources: Prior work shows deep-learning strumming transcription vastly outperforms classical onset detection [35†L409-L417] . We leverage this literature (e.g. Bello & Mayol 2019 [17†L1-L4] , Murgul 2022 [47†L264-L272] , Lukoianov & Klapuri 2025 [45†L38-L45]) and tools (e.g. Librosa onset functions). The recommendations below synthesize these findings into a concrete strategy for your project.

Related Work and Datasets

- **Classical Onset Methods:** Musical onset detection (flux, high-frequency content, complex domain, etc.) is well-studied. Algorithms like spectral flux and phase-based ODF are common [35†L409-L417] . For guitar, specialized tricks (e.g. vibrato suppression) exist [18†L641-L644] . However, pure rule-based methods only achieve ~75–80% F1 on clean guitar signals [35†L409-L417] . Murgul & Heizmann (2022) used Librosa’s spectral flux detector (pruned at 150ms) and achieved $F1 \approx 79\text{--}90\%$ on isolated guitar [20†L43-L50] [47†L264-L272] .
- **Strumming Transcription:** Very recent work targets guitar strums explicitly. Bello & Mayol (2019) built a CNN/LSTM on MFCCs to classify up/down strokes, yielding ~72–70% accuracy [17†L1-L4] . Murgul & Heizmann (2022) combined audio onset detection with a hand-IMU for direction labeling, reaching down/up $F1 \approx 92\%/85\%$ [47†L264-L272] . Janer *et al.* (2025, arXiv) collected 90 min

audio+sensor data and synthesized 4 h of audio, training a CRNN to jointly predict strum onsets, direction, and chords [32†L243-L252] [35†L409-L417] . That model achieved $\approx 97.6\%$ F1 for any-strum detection and $>87\%$ for each direction class [35†L409-L417] . Lukoianov & Klapuri (WASPAA 2025) used a pretrained Transformer (MERT) for strum detection in polyphonic tracks, followed by pattern decoding to an expert vocabulary [45†L38-L45] , achieving “quite high” rhythmic transcription accuracy.

- **Datasets:** There are few large labeled datasets. Murgul’s demo set (ISMIR 2022) is only 5 min (4 chords) [22†L231-L239] [47†L264-L272] . Janer *et al.* (2025) introduce ~ 90 min real + 4 h synthetic. SynthTab (ICASSP 2024) is a 6,700h synthetic guitar tablature dataset [37†L92-L99] (useful for note transcription). *GuitarSet* (Oya *et al.*, 2013) provides multipitch data with hex pickups but not explicit strum labels. For our purposes, we’ll likely need to create a custom dataset (real recordings with manual or semi-auto strum labels, plus synthetic renderings from chord/strum MIDI patterns). Tools like Sonic Visualiser or custom Python scripts can help annotate up/down events.

Dataset	Description	Size/Format	Usage
KLANGIO Multimodal (2022) [47†L264-L272]	5 min guitar with IMU, labeled up/down	WAV + CSV + labels	Baseline testing ($\uparrow$ / $\downarrow$ classification)
Janer <i>et al.</i> StrumData (2025, prep) [32†L243-L252]	90 min real + 4 h synthetic (MIDI→audio)	WAV with onset/direction	CRNN training (onset+direction)
<i>SynthTab</i> (Zang <i>et al.</i> 2024) [37†L92-L99]	6,700 h synthetic guitar (MIDI→audio, many styles)	WAV + tablature	Pretraining for guitar transcription
Proposed (this project)	Real recordings (varied guitars/technique) + synthetic renders	WAV + annotated strums	Model training (focus on strums)

Signal-Processing Features

Key features for strum detection should be **time-domain and spectral onset cues** that are largely pitch-agnostic. Relevant features include:

- **Amplitude envelope & energy:** Sudden rises in the waveform amplitude or energy often correspond to strums. Simple thresholding or envelope-following can detect these transients.
- **Spectral Flux / Novelty:** Compute the change in spectral magnitude between frames (e.g. Euclidean or L2-norm of Mel-spectra diff). Spectral flux is a classic onset function; peaks indicate candidate strums [35†L409-L417] . *SuperFlux* (spectral flux with high-frequency emphasis) is another variant [35†L409-L417] .
- **Complex Domain or Phase Deviation:** Methods like the Complex Domain ODF (which incorporates phase continuity) have shown high recall on guitar onset (98% recall in [35†L409-L417] , although low precision). These can catch weaker attacks that flux misses.

- **High-Frequency Content (HFC):** A weighted sum of high-band magnitudes often spikes at percussive onsets. Guitar pick noise and string attacks have rich high-frequency content.
- **Harmonic-Percussive Source Separation (HPSS):** Separating the signal into “harmonic” (sustained chord) and “percussive” (transient hits) components (e.g. with median filtering) can help. Onset detection applied to the percussive part is often cleaner, since the chord sustain (harmonic) is suppressed. Librosa’s `hpss` or similar can be used here.
- **Temporal Derivatives / Peak Picking:** Compute first derivative of an onset feature (or difference of envelopes) and detect peaks beyond a threshold, possibly with adaptive normalization. Post-processing (e.g. prune onsets too close together, enforce minimum inter-onset interval) is standard.
- **Transients (Time-Domain):** Direct waveform analysis (e.g. short-time RMS derivative, spectral flatness) can capture the impulsive nature of strums. For example, sudden increases in spectral flux or RMS correlate to strum transients.

No single feature is perfect; often combinations are used (see hybrid and ML below). For guitar, string **vibrato** can create false onsets; methods like the “maximum filter” can suppress vibrato fluctuations [18†L641-L644] . Tuning these features is relatively fast computationally, making them good for initial rule-based detectors.

Machine Learning and Deep Learning Approaches

Supervised Models: Given the limited specialized datasets, recent work has turned to deep models trained on synthetic and real audio. Leading approaches include:

- **CNNs on Spectrograms:** Convolutional networks taking Mel-spectrogram or Constant-Q spectrogram inputs. For example, Bello & Mayol (2019) used CNNs on MFCC sequences to classify stroke direction, achieving ~72.5% up/down accuracy [17†L1-L4] . More modern variants use deeper CNN stacks plus recurrent layers.
- **CRNN (Conv+RNN):** Kong *et al.* (2020) introduced a CRNN for piano onset detection; Janer *et al.* adapted this for guitar. Their model (4 conv-blocks + BiGRU) regresses to the time-to-next-onset and classifies up/down [33†L318-L327] [33†L329-L337] . This hybrid regression/classification yields precise onset times beyond fixed frames. Their trained model achieved $F1 \approx 97.6\%$ on pickup signals [35†L409-L417] , far above classical methods.
- **Temporal Convolutional Networks (TCN):** 1D convolution over time (dilated conv) can model longer context. TCNs have succeeded in audio transcription (e.g. “Onsets and Frames” in piano). A TCN could be trained to output onset probabilities (or even direction) per frame.
- **RNNs/LSTMs:** Pure RNNs or GRUs over sequential audio features can capture temporal patterns. However, they often require careful tuning for stability over long sequences.
- **Transformers / Self-Supervised:** Modern audio NLP models (Wav2Vec2, HuBERT, MERT) can be fine-tuned for event detection. Lukoianov & Klapuri (2025) report using a *pretrained foundation model*

(MERT) to detect strums in mixtures [45†L38-L45] . Such models can leverage large unlabeled audio pretraining to improve robustness. In a practical pipeline, one could extract embeddings from a pretrained audio transformer and feed to a classifier/RNN.

- **Sequence Models and Decoders:** Instead of independent frame predictions, sequence-to-sequence or CTC models could transcribe a series of strums. For example, a pattern-decoder (like Klapuri's "pattern-decoding" into vocabulary) can be applied after event detection to enforce a musically valid strumming pattern.
- **Hybrid ML + Rule:** Many approaches combine algorithmic onset detection with ML. For instance, use spectral flux to get candidate times, then classify each candidate as real/false and as down/up with a small CNN or MLP on the local spectrogram slice. This reduces model inference time by focusing only on peaks.

Self-Supervised / Embeddings: While not yet specifically applied to strums, embeddings from models like YAMNet or OpenL3 could be fed into a shallow classifier. Contrastive audio-text models (CLAP) might differentiate "guitar strum" vs "not strum", though specialized training would be needed.

Data Augmentation: Essential for robustness. Janer *et al.* applied pitch-shift (± 6 semitones) [33†L312-L315] , distortion, filters, reverb, and noise [32†L278-L287] . We recommend similar: vary tempo, dynamics, and recording conditions. Synthetic strum generation (using virtual guitars, as in SynthTab [37†L92-L99] or DAWDreamer with AmpleSound) can massively enlarge training data. Balancing real and synthetic data (and possibly human augmentation) yields best results [35†L468-L477] .

Hybrid and Rule-Based Methods

Given constrained data, a **hybrid pipeline** can be effective:

1. **Onset-Only Detection:** First pass with classical onset function (e.g. librosa's `onset_detect` with spectral flux) to get a list of event times. This is fast and language-agnostic. Its recall can be boosted (e.g. using CD-ODF gave ~98% recall [35†L409-L417]), though at cost of false positives.
2. **ML Filtering/Classification:** Use a small CNN or MLP on short spectrogram windows around each candidate to reject spurious onsets and classify down/up. This two-stage approach leverages neural accuracy without per-frame scanning.
3. **Sequence Constraints:** Enforce temporal rules (e.g. align to nearest beat if needed, limit unlikely double hits, prefer downstrums on strong beats). Chord labels can guide this (e.g. strums often coincide with chord changes or metrical emphasis).
4. **Motion Data (if available):** While your project likely has only audio, one could optionally incorporate side-inputs (e.g. if using a microphone that picks up subtle hand noise, or even an accelerometer in a special pick). Such multimodal cues significantly improve direction classification [47†L264-L272] , but are optional.

5. **Quantization/Alignment:** Map detected strums to the known tempo grid from your existing analysis. This not only makes the output tidy for the user but can also help to “snap” onsets that are near beat times, improving perceived accuracy.

Pros & Cons: We summarize in Table 1 the trade-offs of representative methods.

Method	Strengths	Weaknesses	Data/Compute
Spectral Flux / Thresholding	Simple, fast, language-agnostic; no training needed.	Misses weak/fast strums; many false positives. No direction info.	Very low.
Complex Domain ODF	Captures phase + energy changes; high recall [35†L409-L417] .	High false positives; misses quiet onsets; no direction.	Low.
HPC Separation + Onset	Emphasizes attack, reduces sustain. Improves onset SNR.	Extra processing time; quality depends on parameters.	Moderate (but usable offline).
CNN (on MFCC/ Spec)	Learns patterns; can classify direction or string.	Needs labeled data; heavy compute for training.	High training cost, inference moderate.
CRNN (MelSpec + RNN)	Models context; precise timing (regression approach) [33†L318-L327] .	Complex to implement; large data needed to avoid overfit.	High (requires GPU for training, then moderate inference).
Transformer (e.g. MERT)	Leverages large unlabeled audio pretraining [45†L38-L45] ; robust to noise.	Very high compute; may need massive data; risk of overkill.	Very high (heavy model).
Hybrid (rule + ML)	Good trade: uses rule for recall, ML for precision.	More complex pipeline; integration overhead.	Moderate.
Multimodal (audio + IMU)	Excellent direction accuracy [47†L264-L272] .	Requires hardware (not applicable in pure audio pipeline).	NA if no extra sensor.

Table 1: Comparison of strumming detection methods. (Expected F1 in clean, isolated guitar: flux ~75–80%; CRNN ~95–98% [35†L409-L417] .)

Practical Pipeline Integration

We propose the following pipeline to integrate strumming detection into your existing VGT/Reaper workflow:

flowchart TD

```
A[Input Audio (mixed/stem)] --> B{Separate Stems?}
B -->|Yes (LALAL)| C[Guitar Stem (mono)]
B -->|No (already stems)| C
C --> D[Preprocess: downmix, normalize, optional HPSS]
D --> E{Detection Strategy}
E -->|Algorithmic| F1[Compute Onset Function (Flux, CD-ODF, etc.) and Peak-Pick]
E -->|ML Model| F2[Extract features (log-Mel spectrogram) and run CNN/CRNN]
F1 --> G[Candidate Strum Times]
F2 --> G
G --> H[Prune/Refine (min-interval, align to tempo grid)]
H --> I[Classify Direction (e.g. CNN or rule-based)]
I --> J[Quantize to beats/measures]
J --> K[Output Pattern]
K --> L[Export as MIDI notes / Reaper markers / beat-grid events]
```

1. **Stem Separation (if needed):** Use Lalal.ai API or similar to isolate guitar. The isolated stem reduces confusion from other instruments, though artifacts may occur. Our model can be trained on similarly processed audio to be robust.
2. **Preprocessing:** Convert guitar stem to mono, normalize levels. Optionally apply a short HPSS to get percussive signal. Downsample to e.g. 16 kHz to speed up ML (as [33†L306-L314] used).
3. **Onset Detection (Algorithmic):** As a baseline or fallback, compute a novelty function (e.g. `librosa.onset.onset_strength` with flux) and pick peaks above a dynamic threshold. Prune any peaks within ~100–150 ms to avoid double-detecting one strum [20†L69-L73] .
4. **ML-Based Detection:** Prepare features (e.g. log-Mel spectrogram windows; [33†L306-L314]). Feed to a trained model (e.g. CRNN). The model outputs time-to-next-onset for “up” and “down” streams; detect peaks in those regression outputs for events [33†L346-L354] , or simply threshold a final onset probability. This yields a precise list of strum times.
5. **Direction Classification:** If the model outputs up/down directly (as Janer *et al.* did [33†L329-L337]), use it. Otherwise, for each detected strum, extract a short audio snippet and classify up vs. down with a lightweight classifier. (Bello’s CNN on MFCCs or a small 1D CNN on waveform could do this). Alternatively, a simple rule: if the spectral roll-off (or energy distribution) has more high-frequency content after vs. before the event, guess “down”.
6. **Tempo Alignment:** Use the existing tempo grid from your ADT step to snap events to nearest 16th/8th note as appropriate. This yields a clean **strumming pattern** aligned per measure. Minor jitter around the beat is acceptable, but for learner feedback it may be clearer to quantize to an ideal grid (with the original performance time used as guidance).
7. **Output Formatting:** Export the pattern as MIDI or Reaper events. For example, create a new MIDI track where each strum is a note-on: use one MIDI note number (e.g. C2) for downstrums and

another (e.g. C3) for upstrums. This way, the pattern is visible in the piano-roll. Alternatively, generate Reaper region markers at each strum time labeled “↓” or “↑”, or output a MIDI file that can be imported as a *media item*. Beat-grids can be set with Reaper’s tempo map if not already (though your pipeline likely has tempo).

8. **Feedback Generation:** With the pattern detected, you can visually display it in Reaper (or any DAW) aligned with chords and lyrics. This helps students see which beats to strum. You could also compute a “pattern match” score comparing expected vs. played.

Pretrained Tools: Use libraries like Librosa (onset detection, spectrogram), PyTorch/TensorFlow for model. If feasible, start with a lightweight model (e.g. Kong’s CRNN) and PyTorch implementation; it took ~2h training on a V100 for the Janer *et al.* model [33†L380-L383] .

Feature Extraction and Implementation Details

- **Features for ML:** Log-scaled Mel spectrogram (e.g. 229 bins from 30 Hz) was effective [33†L306-L314] . Alternatives: Constant-Q or STFT frames. Avoid pitch-specific features (like chroma) for onset detection.
- **Frame Settings:** [33] used 2048-sample windows with 160-sample hop (~10ms). For hobbyist songs, similar resolution is fine. Adjust hop to ~10ms for precise timing. Apply per-segment normalization to handle varying volume.
- **Data Augmentation:** During training, apply random pitch shifts (± 4 –6 semitones) [33†L312-L315] , dynamic range changes, and typical guitar FX. Use convolutional reverb and additive noise [32†L278-L287] to simulate room/acoustic variations. This aids robustness to recording quality.
- **Model Choices:** A CRNN (CNN+BiGRU) is a strong choice (see Kong *et al.* [33†L295-L303]) and yields high accuracy [35†L409-L417] . A lighter CNN or even a 1D ResNet on raw waveform could be tried for faster inference. A Transformer (like an encoder over spectrogram frames) is promising but may require more data. If limited by compute, start with a 3–4 layer CNN feeding into one GRU layer (as in [33†L319-L327]).
- **Direction Output:** Either have the model output two channels (time-to-up, time-to-down) as Janer *et al.* did [33†L329-L337] , or have it produce a multi-class (up/down/none) per frame. For simplicity, treat direction classification as a separate binary classifier after detecting an onset.
- **Integration with Chord Detection:** Since you already detect chords, you can cross-check: strum events should fall near chord change times or repeated on chord sustains. Use chords as a guide – e.g. if an extra onset is detected right at a chord boundary, it’s likely a downstroke.

Data and Labeling Strategy

1. **Annotation Tools:** For real recordings, tools like **Sonic Visualiser** (onset plugin), **Audacity** (label tracks), or Python scripts with `librosa.onset.onset_detect` (for pre-segmentation) can ease labeling. As in [32], one can align detected onsets with expected metronome beats, then manually

adjust. A simple GUI with waveform+label adjustment (as done by Janer *et al.* [32†L208-L218]) is ideal. For direction, listen or watch video/hand motion if available. Even semi-automated: detect onsets, then label up/down by computing a short-window FFT energy slope (upstrokes often have rising energy).

2. **Synthetic Data:** Generate strumming audio by rendering MIDI/tab with a virtual guitar (e.g. using the SynthTab pipeline with GuitarPro files, or DAWDreamer + AmpleSound VST as in [32†L261-L270]). Create variations in:
 3. **Strum patterns:** 16th/8th patterns (shuffle, syncopation, etc.) drawn from a library [32†L171-L179] .
 4. **Chord progressions:** Major/minor chords, random keys.
 5. **Techniques:** With pick vs finger; mute vs full strum.
 6. **Dynamics:** Vary velocity, drop occasional notes.

Janer *et al.* simulated human imperfections by randomly dropping the last note of a chord 50% of the time [32†L251-L259] . Ensure the synthetic set covers all intended genres/styles (e.g. acoustic vs electric timbres).

1. **Semi-Supervised Labeling:** After training an initial model on synthetic data, use it to pre-label real recordings, then correct mistakes manually. This bootstrapping (active learning) can save time. For instance, run onset detection on student recordings, then have the student or instructor confirm/correct the strum pattern.
2. **Public Sources:** If any YouTube or Creative Commons guitar lessons with separate track audio are available, they could be mined (e.g. backing tracks vs guitar mic). Even without labels, one could run an automatic align of chord+onset patterns between a known score and the audio to create weak labels.

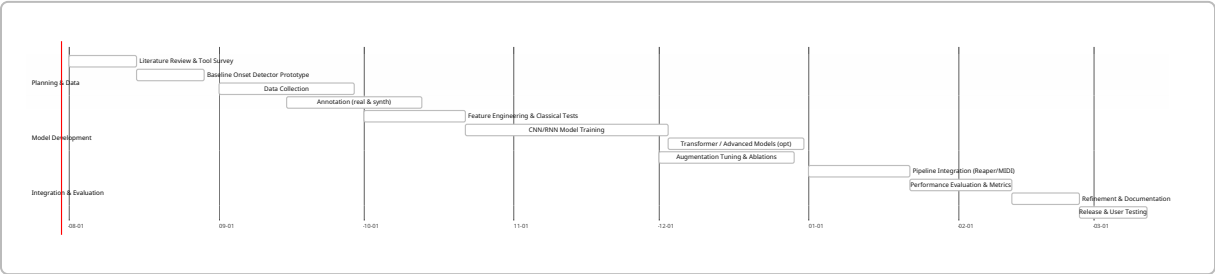
Evaluation Metrics

- **Onset Detection:** Use precision, recall, F1 (with a ± 50 ms tolerance) as per MirEval onset metrics [35†L394-L402] . Report separate F1 for downstrokes and upstrokes.
- **Direction Accuracy:** For each detected onset, check correct up/down label. Compute class-wise F1.
- **Pattern-Level:** If defining a fixed vocabulary of patterns (like Klapuri's method), measure sequence accuracy or edit distance against ground truth pattern sequences.
- **Overall System:** Check integration – e.g. how often a chord symbol in the DAW is aligned with at least one strum event, etc. But primarily focus on onset-level metrics, as these imply pattern correctness.

Include evaluation on both **isolated guitar stems** and on **synthetic mixtures** (if testing mixed-input scenario). Also test on Lalal-separated stems to measure robustness to separation artifacts (which often smear transients). Compare detection rates on raw vs separated audio.

Implementation Plan and Timeline

We outline a staged approach with priorities and mitigate risks at each step. The Gantt chart below summarizes tasks and durations. Key risks include overfitting (mitigated by augmentation and synthetic data), inaccurate labels (semi-automate/peer review), and stem-separation artifacts (augment training with separated-audio).



- **Milestones:** (a1) complete survey and tool testing; (a2) working onset detector; (a3) assembled training set; (a4) trained initial model; (a5) integrated pipeline with measurable output.
- **Risk Mitigation:** If ML model underperforms, fall back to hybrid rule-based pattern (providing “best guess” patterns). Use iterative feedback from hobbyist users: if noise is an issue, advise recording tips. Validate on stems early to ensure separation artifacts are handled.

Conclusion

Automatic guitar strum pattern detection is challenging but feasible with modern techniques. By combining robust onset features with ML classification/regression, and by leveraging synthetic data as in recent literature [32†L243-L252] [37†L92-L99] , we can achieve high accuracy (on par with research prototypes). Our proposed pipeline – from preprocessing to output – builds on proven methods and integrates smoothly with your VGT/Reaper workflow. The tables below summarize key method trade-offs and data resources for reference.